BABU BANARASI DAS UNIVERSITY

2026 - 2027

SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

Student Database Query and CRUD Operations Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA(DS-AI) BCADS25

Submitted by:

Prateek Sharma

Roll No: 06/1250258319

ACKNOWLEDGEMENT

I sincerely thank my teacher Mr. Harendra Singh for guiding me through this NoSQL project titled “Student Database Query and CRUD Operations Using MongoDB”.

While working on this project, I got a clearer understanding of how sorting, comparison operators and logical operators work together inside a query, and how updateOne(), updateMany(), deleteOne() and deleteMany() are used to modify and clean up data in a collection.

I am grateful for the support and guidance provided throughout this project.

INTRODUCTION

This project works with MongoDB and focuses on a set of student records to practise sorting, filtering and CRUD (Create, Read, Update, Delete) operations. A database named college was created, and inside it a collection named student stores the details of students, including their name, age, marks, department and a list of skills.

The main focus of this report is on querying data using comparison operators ($gt, $gte, $lt), combining conditions using logical operators ($or, $and), arranging results using sort(), and finally modifying the collection using updateOne(), updateMany(), deleteOne() and deleteMany().

OBJECTIVES

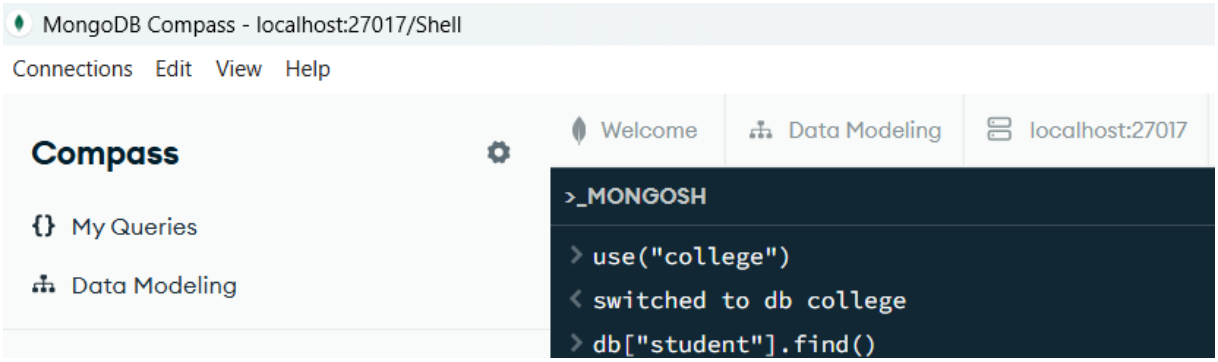
- To connect to a MongoDB database and access a collection.
- To insert multiple student records using insertMany().
- To search for a specific record using find().
- To arrange records in a particular order using sort().
- To filter records using comparison operators — $gt, $gte and $lt.
- To combine multiple conditions using the logical operators $or and $and.
- To modify existing records using updateOne() and updateMany().
- To remove records from a collection using deleteOne() and deleteMany().

1. CONNECT TO THE DATABASE

MongoDB Query:

```
use("college")
```

Explanation: *This command switches to a database named college, creating it if it does not already exist. Once connected, the student collection inside this database is used to store and query all the records used in this project.*

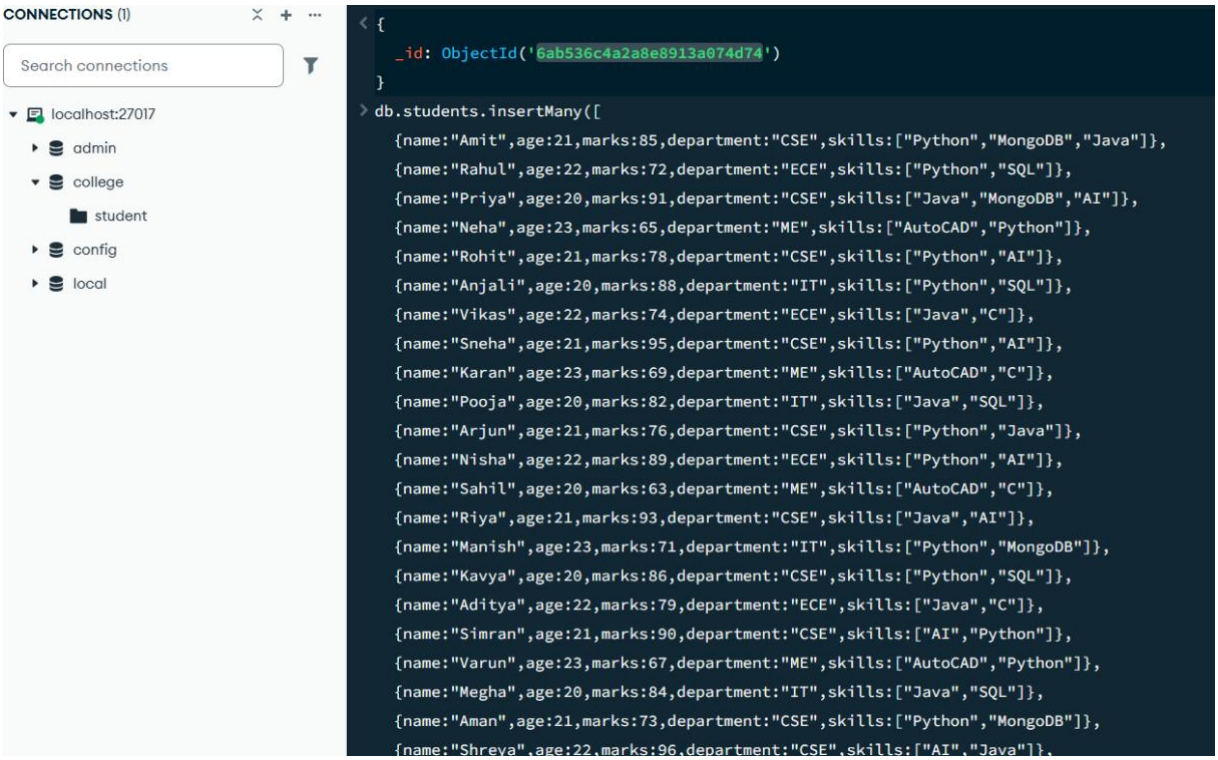


2. INSERT STUDENT RECORDS

MongoDB Query:

```
db.students.insertMany([   { name: "Amit", age: 21, marks: 85, department: "CSE",
skills: ["Python", "MongoDB", "Java"] },   { name: "Rahul", age: 22, marks: 72,
department: "ECE",     skills: ["Python", "SQL"] },   ... and so on for the rest of
the class ])
```

Explanation: *The insertMany() method adds multiple student documents to the students collection in one command. Each document stores the student's name, age, marks, department, and an array of skills they know. This way, all the class records were inserted into the collection at once.*



3. FIND A SPECIFIC RECORD

MongoDB Query:

```
db.students.find({ name: "Priya" })
```

Explanation: *This query searches the collection for a document whose name field matches "Priya" and returns that single record along with all its fields, including the nested skills array and the status field.*

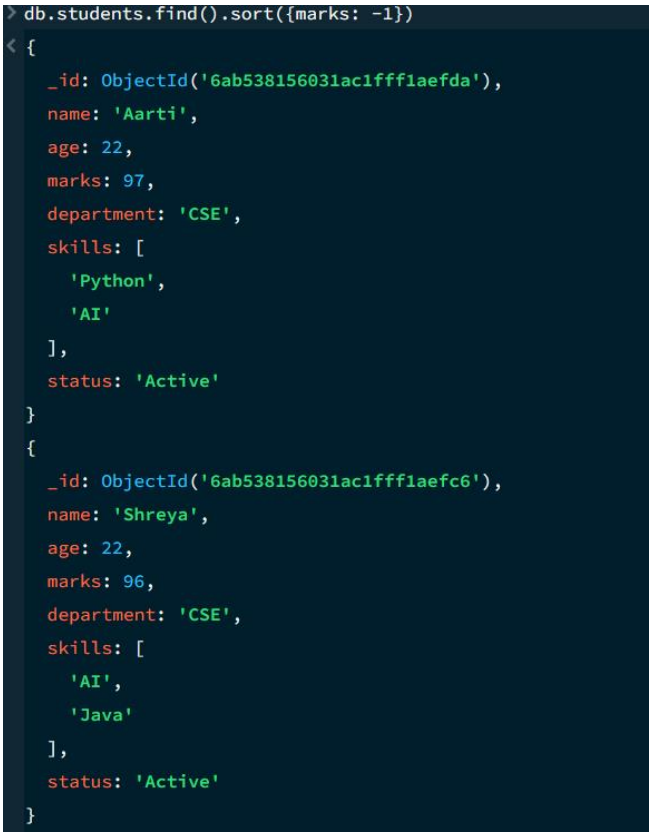


4. SORT RECORDS BY MARKS

MongoDB Query:

```
db.students.find().sort({ marks: -1 })
```

Explanation: *The sort() method arranges the returned documents in a particular order. Passing -1 for the marks field sorts the records in descending order, so the student with the highest marks appears first, followed by the next highest, and so on.*

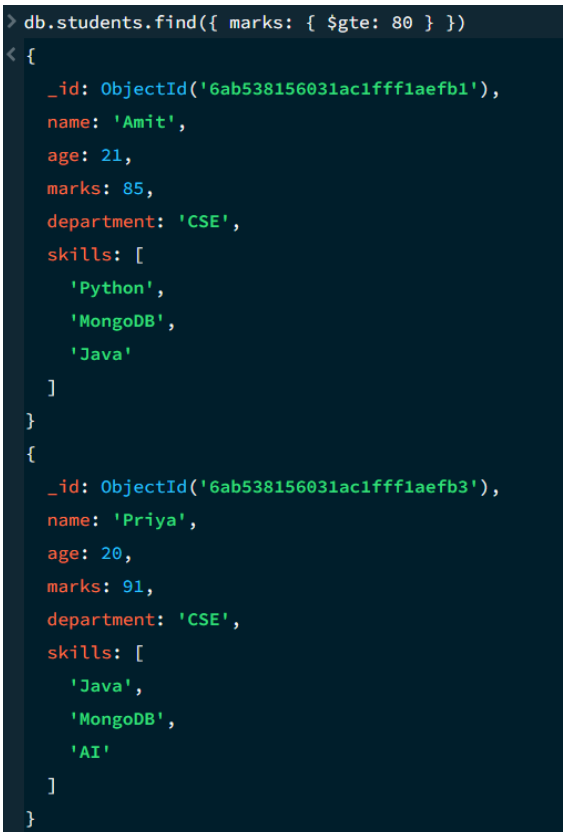


5. GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

```
db.students.find({ marks: { $gte: 80 } })
```

Explanation: *The $gte operator returns documents where the field value is greater than or equal to the given number. This query lists all students who scored 80 marks or above.*



6. GREATER THAN — $gt

MongoDB Query:

```
db.students.find({ marks: { $gt: 80 } })
```

Explanation: *The $gt operator returns documents where the field value is strictly greater than the given number. This query displays students whose marks are above 80, not including a student who scores exactly 80.*

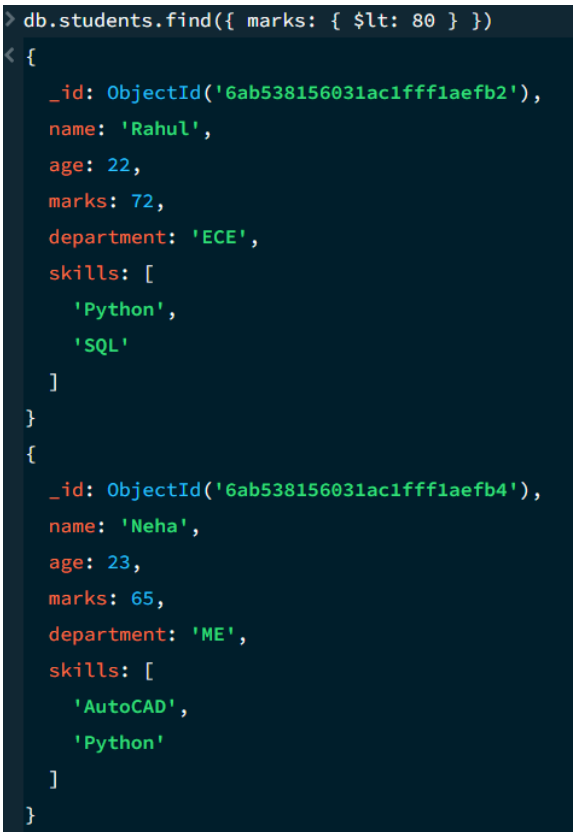


7. LESS THAN — $lt

MongoDB Query:

```
db.students.find({ marks: { $lt: 80 } })
```

Explanation: *The $lt operator returns documents where the field value is less than the given number. This query lists all students whose marks are below 80.*



8. OR OPERATION — DEPARTMENT CSE OR ECE

MongoDB Query:

```
db.students.find({   $or: [     { department: "CSE" },     { department: "ECE" }   ]
})
```

Explanation: *The $or operator returns documents that satisfy at least one of the given conditions. This query finds every student who belongs to either the CSE department or the ECE department.*



9. OR OPERATION — DEPARTMENT CSE OR IT

MongoDB Query:

```
db.students.find({   $or: [     { department: "CSE" },     { department: "IT" }   ] })
```

Explanation: *This is another example of the $or operator, this time checking for students belonging to either the CSE department or the IT department. It shows how the same operator can be reused with a different set of conditions.*

```
> db.students.find({
    $or: [
      {department:"CSE"},
      {department:"IT"}
    ]
  })
{
  _id: ObjectId('6ab538156031ac1fff1aefb1'),
  name: 'Amit',
  age: 21,
  marks: 95,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ],
  status: 'Active'
}
```

10. AND OPERATION — MARKS AND AGE

MongoDB Query:

```
db.students.find({   $and: [     { marks: { $gt: 70 } },     { age: { $lt: 23 } }   ]
})
```

Explanation: *The $and operator returns documents that satisfy all the given conditions together. This query finds students who scored more than 70 marks and are also younger than 23 years — both conditions must hold true at the same time.*



11. AND OPERATION — DEPARTMENT AND MARKS

MongoDB Query:

```
db.students.find({   $and: [     { department: "CSE" },     { marks: { $gt: 90 } }   ]
})
```

Explanation: *This is another use of the $and operator, checking that a student belongs to the CSE department and has scored more than 90 marks. Only records meeting both conditions are returned.*

```
> db.students.find({
    $and: [
      {department:"CSE"},
      {marks:{$gt:90}}
    ]
  })
{
  _id: ObjectId('6ab538156031ac1fff1aefb1'),
  name: 'Amit',
  age: 21,
  marks: 95,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ],
  status: 'Active'
}
```

12. UPDATE OPERATIONS — updateOne() AND updateMany()

MongoDB Query:

```
db.students.updateOne(   { name: "Amit" },   { $set: { marks: 95 } } )
db.students.updateMany(   { department: "CSE" },   { $set: { status: "Active" } } )
```

Explanation: *The updateOne() method updates only the first document that matches the given filter — here, the student named Amit has his marks field changed to 95 using the $set operator. The updateMany() method, on the other hand, updates every document that matches the filter — here, it sets the status field to "Active" for all students belonging to the CSE department. The output confirms both updates through matchedCount and modifiedCount.*



13. DELETE OPERATIONS — deleteOne() AND deleteMany()

MongoDB Query:

```
db.students.deleteOne({ roll: 10 })  db.students.deleteMany({ marks: { $lt: 60 } })
```

Explanation: *The deleteOne() method removes the first document matching the given filter, while deleteMany() removes every document that matches. In this case, neither query found a matching document — no student has a roll field equal to 10, and no student scored below 60 marks — so both operations correctly report a deletedCount of 0, showing that MongoDB safely does nothing when no document matches the filter.*

```
> db.students.deleteOne({roll: 10})
< {
    acknowledged: true,
    deletedCount: 0
  }
> db.students.deleteMany({marks: {$lt: 60}})
< {
    acknowledged: true,
    deletedCount: 0
  }
```

CONCLUSION

This project built on my MongoDB work by focusing on sorting, comparison operators, logical operators and CRUD operations on a student collection. Using MongoDB Compass and the mongosh shell, a set of student records was inserted into a college database and queried in different ways.

I learned how sort() arranges results in ascending or descending order, how $gt, $gte and $lt filter numeric fields, and how $or and $and combine multiple conditions in a single query. I also learned the difference between updateOne() and updateMany(), and between deleteOne() and deleteMany(), including how MongoDB responds when a delete filter matches no documents.

Overall, this project strengthened my practical understanding of query and CRUD operations in MongoDB and gave me more confidence in working with NoSQL databases.